

Secure Pairing-Free IBE and CP-ABE from Inner-Product Functional Encryption

Ahmad Khoureich Ka 

Université Alioune Diop de Bambey, Senegal
ahmadkhoureich.ka@uadb.edu.sn

Abstract. The potential of Attribute-Based Encryption (ABE) in the context of IoT has driven researchers to propose pairing-free ABE schemes that are suitable for resource-constrained devices. Unfortunately, many of these schemes turned out to be insecure. This fact seems to reinforce the point of view of some researchers according to which instantiating an Identity-Based Encryption (IBE) in plain Decisional Diffie-Hellman (DDH) groups is impossible. In this paper, we provide a generic Ciphertext-Policy ABE (CP-ABE) scheme with secret AND gate policy from Inner-Product Functional Encryption (IPFE). We also propose an instantiation of that generic CP-ABE scheme from the DDH assumption. From our generic CP-ABE scheme we derive an IBE scheme by introducing the concept of Clustered Identity-Based Encryption (CIBE). Our schemes show that it is possible to construct secure IBE and ABE schemes based on the classical DDH assumption. An implementation of our CIBE in Python using the Charm framework is available on GitHub.

Keywords: Identity-Based Encryption, Attribute-Based Encryption, Inner-Product Functional Encryption, Decisional Diffie-Hellman.

1 Introduction

Attribute-Based Encryption (ABE) is an asymmetric encryption primitive that allows decryption of the ciphertext if and only if an access policy is satisfied [15,30,5]. The access policy considered as a predicate $P(\cdot)$ is defined on attributes which can be any element that can be used to identify a user. We distinguish two types of ABE. On one side, Key-Policy ABE (KP-ABE) schemes where the access policies are embedded in keys and the set of attributes S is in the ciphertext and on the other side, Ciphertext-Policy ABE (CP-ABE) schemes in which the access-policy $P(\cdot)$ is integrated into the ciphertext and keys are associated to sets of attributes. In either cases decryption is possible only if $P(S) = 1$.

ABE has difficulty penetrating the world of resource-constrained IoT devices because lightweight schemes have the reputation of presenting crippling security flaws [16,17,33] and available secure ABE schemes are resource-intensive. ABE schemes proven secure are based on bilinear pairing [2,29,19] or lattices [10,35,18] and suffer either of large decryption key size or being slow for encryption or decryption. Suitable ABE schemes for lightweight devices [26,27,39,33] exploit

classical security assumption like RSA or the Decisional Diffie-Hellman (DDH), hence their generic name of pairing-free ABE. The discovery of security vulnerabilities in many of those schemes is not surprising to some authors [8,28,17] since they conjectured (*the impossibility conjecture*) that designing Identity-Based Encryption (IBE) schemes that rely on trapdoor permutations or the DDH assumption in a black box manner is impossible. Furthermore, Herranz [17] has claimed that there are many chances that an ABE scheme that works in classical settings (RSA or pairing-free discrete logarithm) is not secure.

1.1 Our Contributions

We provide a generic construction of AND gate access policy CP-ABE from Inner-Product Functional Encryption (IPFE). IPFE has various useful applications including the construction of non-zero inner product encryption [23], the construction of decentralized ABE against bounded collusion [38] and the construction of message selection functional encryption allowing decryption of selected portions of a ciphertext [22].

Our construction is a secret policy CP-ABE scheme. Although the ciphertext hides the message and the encryption policy, our CP-ABE scheme is not attribute-hiding in the sense of Katz *et al.* [24], because an adversary can succeed the indistinguishability experiment where it is asked to guess which of the tuples $(m_0, p_0), (m_1, p_1)$ was encrypted (m_i is the message and p_i is the encryption policy). The secrecy of the access policy and the large policy space make collusion and exhaustive-search attacks infeasible. Secret policy CP-ABE and fully hidden policy CP-ABE provide a better confidentiality than standard CP-ABE. Knowledge of the attributes embedded within the ciphertext can lead to a collapse of the probability distribution on the plaintext space (the set of likely messages a priori shrinks to a much smaller one). To give an example, in wartime intercepting a ciphertext containing attributes referring to a special operations unit of a military branch gives a good chance of guessing the next move of the enemy.

From our generic CP-ABE scheme we derive a *Small Identity Space* IBE (SIS-IBE) that can issue only $\ell - 1$ secret keys where ℓ is the functionality parameter of the underlying IPFE. We introduce the concept of *Clustered* IBE (CIBE) to convert the SIS-IBE scheme to a large identity space IBE scheme.

By providing an instantiation of our CP-ABE scheme from the DDH-based IPFE of [3], we show that it is possible to construct practical and secure ABE and IBE schemes based on the DDH assumption.

1.2 Organization

The rest of this paper is organized as follows. In section 2 we summarize related work. Section 3 presents IPFE and CP-ABE. Our generic CP-ABE scheme is presented in section 4. In section 5, we present a generic IBE scheme. Section 6 concludes this work.

2 Related work

IBE and ABE are useful tools in IoT systems. ABE ensures that only IoT devices that have the required attributes can access sensitive data. IBE simplifies the deployment of public key encryption, as the device's identity can serve as the public key. Many pairing-free ABE schemes suitable for resource-constrained devices have been published. Unfortunately, many of those schemes turned out to be insecure. CP-ABE schemes in [12,37] use scalar multiplication on elliptic curves but they are found vulnerable against collusion attacks [34]. The KP-ABE scheme proposed in [39] is based on the Elliptic Curve Decisional Diffie-Hellman (ECDDH) assumption, but it has been shown in [33] that an adversary can decrypt a ciphertext without having the required attributes. Recently, Hamednejad *et al.* [14] have succeeded in adding two proposals [11,32] into the set of unsecure pairing-free CP-ABE schemes. They have shown that [11,32] are vulnerable to collusion attacks. All these failures tend to reinforce the impossibility conjecture [8,28,17] that says that designing secure IBE schemes that rely solely on the hardness of the Diffie-Hellman Problem is impossible. Nonetheless, attacks against the impossibility conjecture debuted with the work of Döttling and Garg [13] who proposed a construction of IBE under DDH using Garbled Circuits. Thereafter, Blazy and Kakvi [6] provided a construction of IBE similar to that of Döttling and Garg based on Witness Encryption instead of garbled circuits. Unfortunately, these schemes are inefficient and therefore do not completely contradict the impossibility conjecture. In 2024, Yao *et al.* [40] proposed a lightweight pairing-free CP-ABE scheme leveraging the masked authenticated message mechanism of the IOTA protocol to manage authorization. Moreover, Wang *et al.* [36] recently proposed a KP-ABE scheme relying on the DDH assumption. Beside being pairing-free their scheme offers a decentralized architecture.

3 Preliminaries

3.1 Inner-Product Functional Encryption

Inner-Product Functional Encryption allows a recipient who has a secret key derived from a vector $\mathbf{x}$ to obtain from the encryption of a vector $\mathbf{y}$ the inner product $\langle \mathbf{x}, \mathbf{y} \rangle$ and nothing more. Abdalla *et al.* are the first to formalize the notion of IPFE [1]. The syntax of IPFE and its INDistinguishability under Chosen Plaintext Attack (IND-CPA) security are given in Appendix A.

3.2 Identity Based Encryption

Identity Based Encryption was introduced by Shamir in 1984 [31]. IBE is an attractive Public Key Encryption (PKE) primitive since there is no need to tie a random public key to the owner's identity using a certificate management system (as in traditional PKE) because in IBE the user's identity is his or her public key. The syntax of IBE and its indistinguishability-based security are given in Appendix C.

3.3 Ciphertext-policy ABE

A Ciphertext-Policy ABE allows a recipient who has a secret key associated with a set of attributes $\mathbb{S}$ to correctly decrypt a ciphertext integrating an access policy $\mathbb{P}$ if and only if $\mathbb{S}$ satisfies $\mathbb{P}$. In this work, we restrict ourselves to access policies that consist of a single AND gate. Also, our construction is a secret access policy CP-ABE. The syntax of CP-ABE and the indistinguishability-based security for a secret policy CP-ABE scheme are given in Appendix B.

4 Construction of CP-ABE from IPFE

To prevent collusion attacks, our construction is a secret access policy CP-ABE scheme. We use IPFE as building blocks. Let ℓ be the functionality parameter of that IPFE. We denote by $[\ell]$ the set of integers $\{1, \dots, \ell\}$. Let the attribute space be $\mathbb{Z}_q^* \times [\ell]$ for some prime q . Our construction operates on sets of attributes that has at most a cardinality of ℓ where the second coordinates of the attributes are distinct. A set of attributes $\mathbb{S}$ can be represented as a vector $\mathbf{S} \in \mathbb{Z}_q^\ell$. For example, if $\ell = 5$, we represent the set of attributes $\mathbb{S} = \{(12, 3), (25, 5)\}$ as $\mathbf{S} = (0, 0, 12, 0, 25)$. The algorithm for converting a set of attributes $\mathbb{S} = \{(a_i, b_i) \in (\mathbb{Z}_q^* \times [\ell])\}_{i=1}^{n \leq \ell}$ to a vector $\mathbf{S} \in \mathbb{Z}_q^\ell$ is defined as follow:

```

Algorithm toVector( $\mathbb{S}$ )
  Set  $\mathbf{S} = (s_1, \dots, s_\ell) \in \{0\}^\ell$ 
  For each  $(a_i, b_i) \in \mathbb{S}$  do  $s_{b_i} = a_i$  EndFor
  Return  $\mathbf{S}$ 

```

Given a ring $\mathcal{R}$ which is either $\mathbb{Z}$ or $\mathbb{Z}_q$, we denote the coordinate-wise product of two vectors $\mathbf{x}$ and $\mathbf{y}$ in $\mathcal{R}^\ell$ by $\mathbf{x} \odot \mathbf{y} = (x_1 y_1, \dots, x_\ell y_\ell) \in \mathcal{R}^\ell$.

Now, we describe the construction of our generic CP-ABE scheme with secret AND gate policy from from stateful IPFE. The stateless version where the underlying IPFE computes inner products in $\mathbb{Z}$ is syntactically identical but do not require a stateful key generation algorithm.

Setup($1^\lambda, 1^\ell$). Given as input the security parameter λ and the functionality parameter ℓ of the underlying IPFE, this algorithm performs the following steps:

1. Choose a cyclic group $\mathbb{G}$ of prime order $q > 2^\lambda$ with generator g .
2. Call **IPFE.Setup**($1^\lambda, 1^\ell$) to obtain a master secret key msk and a master public key mpk .

KeyGen($msk, st, \mathbb{S}$). Given as input the master secret key msk , the internal state st used by the underlying stateful IPFE and a set of attributes $\mathbb{S} \in (\mathbb{Z}_q^* \times [\ell])^n$ where $n \leq \ell$, this algorithm performs the following steps:

1. Convert the set of attributes $\mathbb{S}$ to a vector $\mathbf{S} = (s_1, \dots, s_\ell) \in \mathbb{Z}_q^\ell$,
2. Call **IPFE.KeyGen**($msk, st, \mathbf{S}$) to obtain a secret key $sk_{\mathbf{S}}$ and an updated state st . Notice that in our scheme, $\mathbf{S}$ is considered secret and is therefore included in $sk_{\mathbf{S}}$.

3. Return (sk_S, st) .

Encrypt $(mpk, \mathbb{P}, m)$. Given as input the master public key mpk , a secret access policy consisting of a set of attributes $\mathbb{P} \in (\mathbb{Z}_q^* \times [\ell])^n$ where $n \leq \ell$ and a message $m \in \mathbb{G}$, this algorithm performs the following steps:

1. Convert the access policy $\mathbb{P}$ to a vector $\mathbf{P} = (p_1, \dots, p_\ell) \in \mathbb{Z}_q^\ell$,
2. Choose a random vector $\mathbf{r} = (r_1, \dots, r_\ell) \in (\mathbb{Z}_q^*)^\ell$.
3. Set $\mathbf{y} = \mathbf{P} \odot \mathbf{r} = (p_1 r_1, \dots, p_\ell r_\ell)$.
4. Compute $c_1 = mg^{\langle \mathbf{P}, \mathbf{y} \rangle}$.
5. Compute $c_2 \leftarrow \text{IPFE.Encrypt}(mpk, \mathbf{y})$.
6. Return the ciphertext $C = (c_1, c_2)$.

Decrypt (mpk, sk_S, C) . Given as input the master public key mpk , a secret key sk_S and a ciphertext $C = (c_1, c_2)$, this algorithm performs the following steps:

1. Compute $\sigma = \text{IPFE.Decrypt}(mpk, sk_S, c_2)$.
2. Return the plaintext $m = c_1 / g^\sigma$.

Correctness. We recall that in an AND gate access policy CP-ABE scheme, $\mathbb{S}$ satisfies $\mathbb{P}$ if $\mathbb{S} \supseteq \mathbb{P}$ meaning that for all $i \in [\ell]$ if $p_i \neq 0$ then $s_i = p_i$. Therefore, $\mathbb{S} \supseteq \mathbb{P} \iff \mathbf{S} \odot \mathbf{P} = \mathbf{P} \odot \mathbf{P}$. The correctness of the scheme follows from the observation that:

$$\begin{aligned} \sigma &= \text{IPFE.Decrypt}(mpk, sk_S, c_2) \\ &= \text{IPFE.Decrypt}(mpk, sk_S, \text{IPFE.Encrypt}(mpk, \mathbf{y})) \\ &= \langle \mathbf{S}, \mathbf{y} \rangle = \langle \mathbf{S}, \mathbf{P} \odot \mathbf{r} \rangle = \langle \mathbf{S} \odot \mathbf{P}, \mathbf{r} \rangle = \langle \mathbf{P} \odot \mathbf{P}, \mathbf{r} \rangle \\ &= \langle \mathbf{P}, \mathbf{P} \odot \mathbf{r} \rangle = \langle \mathbf{P}, \mathbf{y} \rangle \end{aligned}$$

Therefore, $c_1 / g^\sigma = c_1 / g^{\langle \mathbf{P}, \mathbf{y} \rangle} = m$.

4.1 Security

We recall that in our setting the encryptor keeps the access policy secret and the set of attributes associated with a secret key is also secret. These requirements are necessary to ensure that collusion attacks are infeasible. If the access policy is known, the adversary can find vectors that do not satisfy the access policy but for which there exists a linear combination that does satisfy it. The adversary can make queries for secret keys associated to these vectors. By decrypting c_2 (the IPFE component of the ciphertext) with each of these secret keys and summing the set of obtained numbers, the adversary recovers the secret $\langle \mathbf{P}, \mathbf{y} \rangle$ used to mask the plaintext. The same reasoning applies when the set of attributes associated with a secret key that can decrypt the ciphertext is discovered. With these requirements in mind, the security of the underlying IPFE scheme reduces to the security of our CP-ABE scheme.

Theorem 1. *If the underlying IPFE scheme is IND-CPA secure, then our CP-ABE scheme is also IND-CPA secure. (The proof is presented in Appendix D)*

4.2 Instantiation from the DDH Assumption

We present an instantiation of our generic CP-ABE scheme from a modified version of the DDH-based IPFE scheme of [3] that we denote IPFE-DDH*. The modification is only made to the decryption algorithm to return $g_1^{\langle \mathbf{x}, \mathbf{y} \rangle}$ instead of $\langle \mathbf{x}, \mathbf{y} \rangle$ thus avoiding the computation of the discrete logarithm which is too expensive. IPFE-DDH* is presented in Appendix E. The KeyGen algorithm of this instantiation proceeds exactly as in the generic construction in Section 4. Therefore, we only present algorithms (or steps) that change.

Setup($1^\lambda, 1^\ell$)

2. Call IPFE-DDH.Setup($1^\lambda, 1^\ell$) to obtain the master secret key msk and a master public key $mpk = (\mathbb{G}, g_1, g_2, \{h_i\}_{i=1}^\ell)$.

Encrypt($mpk, \mathbb{P}, m$)

4. Compute $c_1 = mg_1^{\langle \mathbf{P}, \mathbf{y} \rangle}$.

Decrypt(mpk, sk_S, C)

1. Compute $\rho = \text{IPFE-DDH.Decrypt}(mpk, sk_S, c_2)$
2. Return the plaintext $m = c_1 / \rho$.

4.3 Theoretical Evaluation

We do a theoretical comparison of our DDH-based CP-ABE scheme with other schemes such as FAME [2], FABEO [29] and Easy-ABE [19]. All these schemes are pairing based ABE schemes. This comparison is made only on AND gate access policies (the only one supported by our scheme). FAME and FABEO support access policies described by boolean formulas that are convertible into Monotone Span Programs (MSPs). An MSPs consists of a matrix M and a mapping π that assigns each row of M to an attribute. For many ABE schemes, it is required that π be injective we say that those schemes have one-use restriction. Techniques [25] to avoid the one-use restriction can be used but lead to a less efficient scheme [29]. Easy-ABE does not use MSPs (therefore does not suffer from the one-use restriction), it derives from the boolean formula consisting of AND/OR gates the collection of authorized sets of attributes and then converts each authorized set into a bit string using the universe of attributes. In this comparison (see Tables 1, 2 and 3), we use the one-use restriction versions of FAME and FABEO.

Some may think that comparing our CP-ABE scheme against pairing-based CP-ABE ones supporting more complex policies is unfair. Nonetheless, this comparison gives a glimpse of the behavior of our scheme in the setting of AND gate access policy.

Table 1. Number of group operations used for key generation. T denotes the number of attributes input to the KeyGen algorithm, ℓ is the functionality parameter of the DDH-based IPFE.

Scheme	Key generation			
	$\mathbb{G}_1$ or $\mathbb{Z}_q$ for our			$\mathbb{G}_2$
	Mul	Exp	Hash	Exp
FAME	$8T + 9$	$9T + 9$	$6(T + 1)$	3
FABEO	1	$T + 2$	$T + 1$	1
Easy-ABE	1	2	1	1
Our	2ℓ	—	—	—

Table 2. Number of group operations used for encryption and decryption. n_1, n_2 are the number of rows and columns of the MSPs in FAME and FABEO, I is the number of attributes used in decryption, m is the number of attribute strings of the access policy in Easy-ABE, ℓ is the functionality parameter of the DDH-based IPFE.

Scheme	Encryption				Decryption			
	$\mathbb{G}_1$			$\mathbb{G}_2$	$\mathbb{G}_1$		$\mathbb{G}_T$	
	Mul	Exp	Hash	Exp	Mul	Exp	Mul	Pairing
FAME	$12n_1n_2 + 6n_1$	$6n_1$	$6(n_1 + n_2)$	3	$6I + 3$	—	6	6
FABEO	n_1	$2n_1$	$n_1 + 1$	2	$2I$	—	3	3
Easy-ABE	1	$m + 2$	m	1	—	—	1	2
Our	$2\ell + 1$	$2\ell + 3$	—	—	$\ell + 2$	$\ell + 2$	—	—

Table 3. Size of keys and ciphertexts. T denotes the number of attributes input to the KeyGen algorithm, n_1 is the number of rows of the MSPs in FAME and FABEO, m is the number of attribute strings of the access policy in Easy-ABE, ℓ is the functionality parameter of the DDH-based IPFE.

Scheme	Storage cost			
	Key size		Ciphertext size	
	$\mathbb{G}_1$ or $\mathbb{Z}_q$ for our	$\mathbb{G}_2$	$\mathbb{G}_1$	$\mathbb{G}_2$
FAME	$3(T + 1)$	3	$3n_1$	3
FABEO	$T + 1$	1	n_1	2
Easy-ABE	1	1	$m + 1$	1
Our	$\ell + 2$	—	$\ell + 3$	—

4.4 Experimental Evaluation

We implemented our DDH-based CP-ABE scheme using the Charm 0.5 framework [4]. The implementations of FAME [2], FABEO [29] and Easy-ABE [19] are taken from github [20]. All the schemes are run on an Ubuntu 22.04 desktop computer with an Intel Core i5-4590S CPU and 8GB RAM. The pairing based schemes (FAME, FABEO, Easy-ABE) use the MNT224 curve and our scheme uses the group $\mathbb{G}_1$ of MNT224 of the Charm framework.

The experiments are carried out for the four algorithms of each scheme. For FAME, FABEO and Easy-ABE, we use AND gate access policies of the form "1 AND 2 AND ... AND N " where attributes are in $[N]$ for $N \in \{10, 20, \dots, 100\}$ as in [2,29,19]. For our DDH-based CP-ABE scheme we use the same form of access policies but attributes are big integers randomly picked from the integer group $\mathbb{Z}_R$ of the Charm framework. Indeed, the size of the numbers used as attributes has an effect on the execution times of our scheme, and has no effect on the other schemes. Results are obtained by averaging over 20 executions.

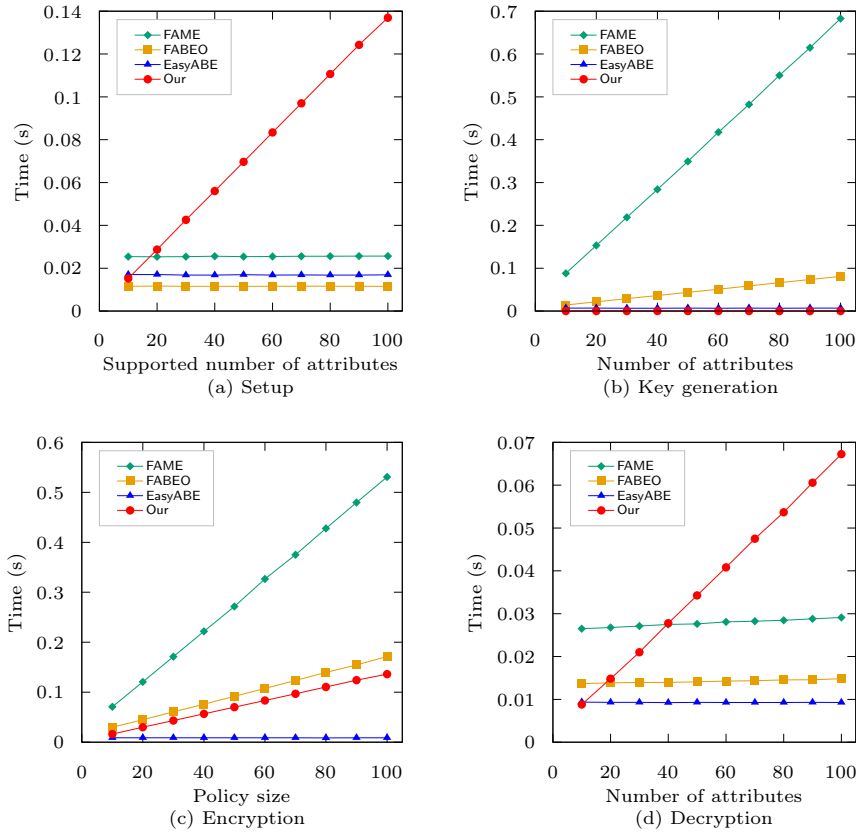


Fig. 1. Average running times for the setup, key generation, encryption and decryption algorithms with AND gate access policies. The pairing based schemes support public and expressive policies while our DDH-based CP-ABE only supports AND-gates policy. This comparison serves only to have a reference for the order of magnitude of the execution times of our scheme.

Figure 1 shows how the running times of the setup, key generation, encryption, and decryption algorithms scale with the size of the access policy or the number of attributes.

Table 4. Size of keys and ciphertexts for 100 attributes with the AND gate policy. Ciphertext sizes are obtained by encrypting a random plaintext of 1024 bytes.

Scheme	Storage cost (bytes)	
	Key size	Ciphertext size
FAME	20800	23322
FABEO	7695	10300
Easy-ABE	435	2261
Our	6762	9050

Note that table 4 that gives the storage cost of the different schemes is inline with table 3.

5 The case $n = \ell$ in our CP-ABE scheme

In our CP-ABE construction, the AND gate access policy is a set of attributes, $\mathbb{P} \in (\mathbb{Z}_q^* \times [\ell])^n$ where $n \leq \ell$. The case $n = \ell$ allows us to derive an Identity-Based Encryption scheme that we construct in two stages. First, we construct in section 5.1 a *small identity space* IBE that (to be secure) can only issue a small number of secret keys. Next, we overcome this limitation in section 5.2 by introducing the notion of *clustered* IBE which consists of grouping many instances of our small identity space IBE. The resulting scheme has a large id space and is secure against adaptive chosen plaintext attacks.

5.1 Small ID Space Identity-Based Encryption

As its name suggests, this IBE scheme can only issue a small number of secret keys. Therefore, the underlying IPFE does not need to be stateful since less than ℓ (its functionality parameter) keys are produced.

Setup($1^\lambda, 1^\ell$). λ is the security parameter and ℓ is the functionality parameter of the underlying IPFE. The setup algorithm performs the following steps:

1. Choose a cyclic group $\mathbb{G}$ of prime order $q > 2^\lambda$ with generator g .
2. Run $(mpk, msk) \leftarrow \text{IPFE.Setup}(1^\lambda, 1^\ell)$.
3. Create a list $\mathbb{L}$ (initially empty) of vectors in $(\mathbb{Z}_q^*)^\ell$. $\mathbb{L}$ has a maximum capacity of $\ell - 1$ vectors.
4. Define $params = (\mathbb{G}, g, mpk, \mathbb{L})$.

KeyGen($params, msk, \text{ID}$). Given $\text{ID} \in [\ell - 1]$, this algorithm performs the following steps:

1. Check if the vector $\mathbf{x}_{\text{ID}}$ is in $\mathbb{L}$. $\mathbf{x}_{\text{ID}}$ is the random vector in $(\mathbb{Z}_q^*)^\ell$ associated to ID.
2. If yes, run $sk_{\mathbf{x}_{\text{ID}}} \leftarrow \text{IPFE.KeyGen}(msk, \mathbf{x}_{\text{ID}})$ and return $sk_{\mathbf{x}_{\text{ID}}}$.
3. Otherwise:
 - 3.1. Pick a random $\mathbf{x}_{\text{ID}} \in (\mathbb{Z}_q^*)^\ell$ such that the set of vectors $\{\mathbf{x}\}_{\mathbf{x} \in \mathbb{L}} \cup \{\mathbf{x}_{\text{ID}}\}$ is linearly independent, and add $\mathbf{x}_{\text{ID}}$ to $\mathbb{L}$.
 - 3.2. Run $sk_{\mathbf{x}_{\text{ID}}} \leftarrow \text{IPFE.KeyGen}(msk, \mathbf{x}_{\text{ID}})$ and return $sk_{\mathbf{x}_{\text{ID}}}$.

Encrypt($params, \text{ID}, m$). Given $\text{ID} \in [\ell - 1]$ and $m \in \mathbb{G}$, this algorithm performs the following steps:

1. Check if $\mathbf{x}_{\text{ID}}$ is in $\mathbb{L}$.
2. If no then abort. We require that encryption is only possible for identities for which a secret key is already produced.
3. Otherwise:
 - 3.1. $\mathbf{x}_{\text{ID}} = (x_1, \dots, x_\ell) \in \mathbb{L}$.
 - 3.2. Choose a random vector $\mathbf{r} = (r_1, \dots, r_\ell) \in (\mathbb{Z}_q^*)^\ell$.
 - 3.3. Set $\mathbf{y} = \mathbf{x}_{\text{ID}} \odot \mathbf{r} = (x_1 r_1, \dots, x_\ell r_\ell)$.
 - 3.4. Compute $c_1 = mg^{\langle \mathbf{x}_{\text{ID}}, \mathbf{y} \rangle}$.
 - 3.5. Compute $c_2 \leftarrow \text{IPFE.Encrypt}(mpk, \mathbf{y})$.
 - 3.6. Return the ciphertext $C = (c_1, c_2)$.

Decrypt($params, sk_{\mathbf{x}_{\text{ID}}}, C$). Given a secret key $sk_{\mathbf{x}_{\text{ID}}}$ and a ciphertext $C = (c_1, c_2)$ this algorithm performs the following steps:

1. Compute $\sigma = \text{IPFE.Decrypt}(mpk, sk_{\mathbf{x}_{\text{ID}}}, c_2)$.
2. Return the plaintext $m = c_1 / g^\sigma$.

Security Intuitively the restriction on the size of the identity space ($\ell - 1$ identities corresponding to $\ell - 1$ linearly independent vectors) guarantees to our SIS-IBE scheme resistance against collusion attacks and prevents an adversary to recover $\mathbf{y}$ from c_2 . Therefore, the security of the underlying IPFE scheme reduces to the security of our SIS-IBE scheme.

Theorem 2. *If the underlying IPFE scheme is IND-CPA secure, then our small identity space IBE scheme is also IND-CPA secure. (The proof is presented in Appendix F)*

5.2 Extensions

Our SIS-IBE scheme can only support $\ell - 1$ identities where ℓ is the functionality parameter of the underlying IPFE scheme. This limitation can be overcome by grouping multiple instances of SIS-IBE. We call the resulting scheme a *Clustered Identity-Based Encryption*. An implementation in Python is available on GitHub [21]. Hereafter is a description of its construction.

Setup($1^\lambda, 1^\ell$). λ is the security parameter and ℓ is the functionality parameter of the underlying IPFE. The setup algorithm performs the following steps:

1. Choose a cyclic group $\mathbb{G}$ of prime order $q > 2^\lambda$ with generator g .
2. Define two empty lists denoted MPK (of master public keys) and MSK (of master secret keys).
3. Create a set of lists $\{\mathbb{L}_\alpha\}_{\alpha \in \mathbb{Z}}$ initially empty. Each $\mathbb{L}_\alpha$ is initially empty and has a maximum capacity of $\ell - 1$ vectors.
4. Define $params = (1^\lambda, 1^\ell, \mathbb{G}, g, \text{MPK}, \{\mathbb{L}_\alpha\}_{\alpha \in \mathbb{Z}})$.

KeyGen($params, \text{MSK}, \text{ID}$). Given $\text{ID} \in \mathbb{Z}$, this algorithm performs the following steps:

1. Compute $\alpha = \lfloor \frac{\text{ID}}{\ell-1} \rfloor$ and $\beta = \text{ID} \bmod (\ell - 1)$.
2. Check if msk_α is in MSK.
3. If no, run $(mpk_\alpha, msk_\alpha) \leftarrow \text{IPFE.Setup}(1^\lambda, 1^\ell)$, add mpk_α to MPK and msk_α to MSK.
4. Check if the vector $\mathbf{x}_\beta$ is in $\mathbb{L}_\alpha$.
5. If yes, run $sk_{\mathbf{x}_\beta} \leftarrow \text{IPFE.KeyGen}(msk_\alpha, \mathbf{x}_\beta)$, set $sk_{\mathbf{x}_\beta} = (sk_{\mathbf{x}_\beta}, \alpha)$ and return $sk_{\mathbf{x}_\beta}$.
6. Otherwise:
 - 6.1. Pick a random $\mathbf{x}_\beta \in (\mathbb{Z}_q^*)^\ell$ such that the set of vectors $\{\mathbf{x}\}_{\mathbf{x} \in \mathbb{L}_\alpha} \cup \{\mathbf{x}_\beta\}$ is linearly independent and add $\mathbf{x}_\beta$ to $\mathbb{L}_\alpha$.
 - 6.2. Run $sk_{\mathbf{x}_\beta} \leftarrow \text{IPFE.KeyGen}(msk_\alpha, \mathbf{x}_\beta)$, set $sk_{\mathbf{x}_\beta} = (sk_{\mathbf{x}_\beta}, \alpha)$ and return $sk_{\mathbf{x}_\beta}$.

Note that the clusters are created by **KeyGen**. A cluster consists of a master key pair (msk, mpk) and a set of $\ell - 1$ linearly independent vectors. A given ID in $\mathbb{Z}$ is mapped to a tuple $(\alpha, \mathbf{x})$ where $\mathbf{x} \in (\mathbb{Z}_q^*)^\ell$ is a vector of the cluster numbered α .

Encrypt($params, \text{ID}, m$). Given $\text{ID} \in \mathbb{Z}$ and $m \in \mathbb{G}$, this algorithm performs the following steps:

1. Compute $\alpha = \lfloor \frac{\text{ID}}{\ell-1} \rfloor$ and $\beta = \text{ID} \bmod (\ell - 1)$.
2. Check if $\mathbf{x}_\beta = (x_1, \dots, x_\ell)$ is in $\mathbb{L}_\alpha$.
3. If no then abort. We require that encryption is only possible for identities for which a secret key is already produced.
4. Otherwise: (mpk_α necessarily exists in MPK)
 - 4.1. Choose a random vector $\mathbf{r} = (r_1, \dots, r_\ell) \in (\mathbb{Z}_q^*)^\ell$.
 - 4.2. Set $\mathbf{y} = \mathbf{x}_\beta \odot \mathbf{r} = (x_1 r_1, \dots, x_\ell r_\ell)$.
 - 4.3. Compute $c_1 = mg^{\langle \mathbf{x}_\beta, \mathbf{y} \rangle}$.
 - 4.4. Compute $c_2 \leftarrow \text{IPFE.Encrypt}(mpk_\alpha, \mathbf{y})$.
 - 4.5. Return the ciphertext $C = (c_1, c_2)$.

Decrypt($params, sk_{\mathbf{x}}, C$). Given a ciphertext $C = (c_1, c_2)$ and a secret key $sk_{\mathbf{x}}$ (containing α), this algorithm performs the following steps:

1. Compute $\sigma = \text{IPFE.Decrypt}(mpk_\alpha, sk_{\mathbf{x}}, c_2)$.
2. Return the plaintext $m = c_1 / g^\sigma$.

Note that the size of the master key pair (MPK, MSK) grows linearly as the number of clusters grows but the instantiation using a DDH based IPFE such as of [3,9] can give small and constant-size secret keys.

The encryption algorithm fails if we try to encrypt an identity that does not yet have a decryption key. This limitation does not make our proposal impractical. It can be used in web-based email services where users get their decryption key at the creation of their account. It can also be deployed in IoT systems where devices get their decryption key at their registration with the IoT infrastructure. In these cases, it is guaranteed that the recipient for whom we are encrypting has a decryption key. Therefore, the encryption algorithm will not abort.

The key generation algorithm is responsible for creating clusters and maintaining the list of vectors in each cluster. Therefore, this algorithm is stateful. Requiring to a stateful key generation algorithm has already been done in [3,9] to make their IPFE schemes computing inner products in $\mathbb{Z}_q$ operational. This design choice raises some issues, namely: secure storage of master secret keys, search efficiency for **KeyGen** and **Encrypt**, and communication overhead. Nonetheless, a good choice of the clusters size can mitigate these problems because the larger the clusters size, the fewer there are.

Security. We state the IND-CPA security of our CIBE scheme by the following theorem with an overview of its proof.

Theorem 3. *If the underlying IPFE scheme is IND-CPA secure, then our clustered IBE scheme is also IND-CPA secure.*

Proof (Proof overview). This theorem can be proved by reduction. One can convert an instance of our CIBE scheme into an instance of the SIS-IBE scheme and prove that an adversary attacking our CIBE scheme has the same advantage as an adversary attacking our SIS-IBE scheme. Therefore, with theorem 2 one can conclude with the statement of theorem 3.

By using a clustering technique, we converted our SIS-IBE scheme into a scalable and much more efficient IBE scheme. To give an example, a SIS-IBE scheme that can produce 10000 secret keys ($\ell = 10001$) can be converted into a clustered IBE consisting of 477 clusters each producing 21 secret keys ($\ell = 22$). In this context, the sizes of the master keys of the two schemes (SIS-IBE and CIBE instantiated from the DDH assumption) are roughly the same but the clustered IBE is scalable and provides a better user experience because it enjoys a smaller secret key storage cost and its encryption and decryption processes are much more efficient (Figures 1.c and 1.d give a glimpse of how the running times for the encryption and decryption algorithms scale with ℓ).

6 Conclusion

We have used IPFE as building blocks to construct a generic secure AND gate CP-ABE scheme. A practical implementation from the DDH assumption is also provided. A theoretical and experimental evaluation of our proposal has been

done. From our generic CP-ABE construction we derived a *clustered* IBE scheme. Although the latter scheme may suffer from large master keys (as the number of clusters increases), on the user side a small and constant-size secret key can be enjoyed. Additionally, an appropriate cluster size provides acceptable execution time for the decryption algorithm. Our constructions are non-black-box; therefore, they do not contradict the impossibility conjecture but they prove that it is possible to construct practical and secure IBE and ABE schemes based on the DDH assumption.

Acknowledgments

We thank the anonymous reviewers for their constructive comments and valuable suggestions.

References

1. Abdalla, M., Bourse, F., De Caro, A., Pointcheval, D.: Simple functional encryption schemes for inner products. In: Katz, J. (ed.) Public-Key Cryptography – PKC 2015. pp. 733–751 (2015)
2. Agrawal, S., Chase, M.: Fame: Fast attribute-based message encryption. In: Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security. p. 665–682 (2017). <https://doi.org/10.1145/3133956.3134014>
3. Agrawal, S., Libert, B., Stehlé, D.: Fully secure functional encryption for inner products, from standard assumptions. In: Robshaw, M., Katz, J. (eds.) Advances in Cryptology – CRYPTO 2016. pp. 333–362 (2016)
4. Akinyele, J.A., Garman, C., Miers, I., Pagano, M.W., Rushanan, M., Green, M., Rubin, A.D.: Charm: a framework for rapidly prototyping cryptosystems. Journal of Cryptographic Engineering pp. 111–128 (2013). <https://doi.org/10.1007/s13389-013-0057-3>
5. Bethencourt, J., Sahai, A., Waters, B.: Ciphertext-policy attribute-based encryption. In: IEEE Symposium on Security and Privacy – SP 2007. pp. 321–334 (2007). <https://doi.org/10.1109/SP.2007.11>
6. Blazy, O., Kakvi, S.A.: Identity-based encryption in DDH hard groups. In: Batina, L., Daemen, J. (eds.) Progress in Cryptology – AFRICACRYPT 2022. pp. 81–102 (2022)
7. Boneh, D., Franklin, M.: Identity-based encryption from the weil pairing. In: Kilian, J. (ed.) Advances in Cryptology – CRYPTO 2001. pp. 213–229 (2001)
8. Boneh, D., Papakonstantinou, P., Rackoff, C., Vahlis, Y., Waters, B.: On the impossibility of basing identity based encryption on trapdoor permutations. In: 2008 49th Annual IEEE Symposium on Foundations of Computer Science. pp. 283–292 (2008). <https://doi.org/10.1109/FOCS.2008.67>
9. Castagnos, G., Laguillaumie, F., Tucker, I.: Practical fully secure unrestricted inner product functional encryption modulo p . In: Peyrin, T., Galbraith, S.D. (eds.) Advances in Cryptology – ASIACRYPT 2018. pp. 733–764 (2018)
10. Dai, W., Doröz, Y., Polyakov, Y., Rohloff, K., Sajjadpour, H., Savaş, E., Sunar, B.: Implementation and evaluation of a lattice-based key-policy ABE scheme. IEEE Transactions on Information Forensics and Security pp. 1169–1184 (2018). <https://doi.org/10.1109/TIFS.2017.2779427>

11. Das, S., Namasudra, S.: Multiauthority cp-abe-based access control model for iot-enabled healthcare infrastructure. *IEEE Transactions on Industrial Informatics* pp. 821–829 (2023). <https://doi.org/10.1109/TII.2022.3167842>
12. Ding, S., Li, C., Li, H.: A novel efficient pairing-free CP-ABE based on elliptic curve cryptography for IoT. *IEEE Access* pp. 27336–27345 (2018). <https://doi.org/10.1109/ACCESS.2018.2836350>
13. Döttling, N., Garg, S.: Identity-based encryption from the diffie-hellman assumption. In: Katz, J., Shacham, H. (eds.) *Advances in Cryptology – CRYPTO 2017*. pp. 537–569 (2017)
14. Farnoosh, H., Javad, M., A., M.R.: Attacking two pairing-free ciphertext-policy attribute-based encryption schemes. *The ISC International Journal of Information Security (ISecure)* p. 151–160 (2025). <https://doi.org/10.22042/isecure.2025.216447>
15. Goyal, V., Pandey, O., Sahai, A., Waters, B.: Attribute-based encryption for fine-grained access control of encrypted data. In: *Proceedings of the 13th ACM Conference on Computer and Communications Security*. p. 89–98. CCS '06 (2006)
16. Herranz, J.: Attribute-based encryption implies identity-based encryption. *IET Information Security* pp. 332–337 (2017). <https://doi.org/10.1049/iet-ifs.2016.0490>
17. Herranz, J.: Attacking pairing-free attribute-based encryption schemes. *IEEE Access* pp. 222226–222232 (2020). <https://doi.org/10.1109/ACCESS.2020.3044143>
18. Huang, B., Gao, J., Li, X.: Efficient lattice-based revocable attribute-based encryption against decryption key exposure for cloud file sharing. *Journal of Cloud Computing* p. 37 (2023). <https://doi.org/10.1186/s13677-023-00414-w>
19. Ka, A.K.: Easy-ABE: An easy ciphertext-policy attribute-based encryption. In: Bella, G., Doinea, M., Janicke, H. (eds.) *Innovative Security Solutions for Information Technology and Communications*. pp. 168–183 (2023)
20. Ka, A.K.: EasyABE github (2023), <https://github.com/khoureich/EasyABE>
21. Ka, A.K.: CIBE github (2024), <https://github.com/khoureich/Clustered-IBE>
22. Ka, A.K.: M-Sel: A message selection functional encryption from simple tools. In: Manulis, M., Maimuṭ, D., Teşeleanu, G. (eds.) *Innovative Security Solutions for Information Technology and Communications*. pp. 79–96 (2024)
23. Katsumata, S., Yamada, S.: Non-zero inner product encryption schemes from various assumptions: LWE, DDH and DCR. In: Lin, D., Sako, K. (eds.) *Public-Key Cryptography – PKC 2019*. pp. 158–188 (2019)
24. Katz, J., Sahai, A., Waters, B.: Predicate encryption supporting disjunctions, polynomial equations, and inner products. In: Smart, N. (ed.) *Advances in Cryptology – EUROCRYPT 2008*. pp. 146–162 (2008)
25. Kowalczyk, L., Wee, H.: Compact adaptively secure ABE for NC^1 from k-lin. In: Ishai, Y., Rijmen, V. (eds.) *Advances in Cryptology – EUROCRYPT 2019*. pp. 3–33 (2019)
26. Odelu, V., Das, A.K.: Design of a new CP-ABE with constant-size secret keys for lightweight devices using elliptic curve cryptography. *Sec. and Commun. Netw.* p. 4048–4059 (2016). <https://doi.org/10.1002/sec.1587>
27. Odelu, V., Das, A.K., Khurram Khan, M., Choo, K.K.R., Jo, M.: Expressive CP-ABE scheme for mobile devices in IoT satisfying constant-size keys and ciphertexts. *IEEE Access* pp. 3273–3283 (2017). <https://doi.org/10.1109/ACCESS.2017.2669940>
28. Papakonstantinou, P.A., Rackoff, C.W., Vahlis, Y.: How powerful are the DDH hard groups? *Cryptology ePrint Archive, Paper 2012/653* (2012)

29. Riepel, D., Wee, H.: Fabeo: Fast attribute-based encryption with optimal security. In: Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security. p. 2491–2504 (2022). <https://doi.org/10.1145/3548606.3560699>
30. Sahai, A., Waters, B.: Fuzzy identity-based encryption. In: Cramer, R. (ed.) Advances in Cryptology – EUROCRYPT 2005. pp. 457–473 (2005)
31. Shamir, A.: Identity-based cryptosystems and signature schemes. In: Blakley, G.R., Chaum, D. (eds.) Advances in Cryptology. pp. 47–53 (1985)
32. Sowjanya, K., Dasgupta, M.: A ciphertext-policy attribute based encryption scheme for wireless body area networks based on ecc. Journal of Information Security and Applications **54**, 102559 (2020). <https://doi.org/10.1016/j.jisa.2020.102559>
33. Tan, S.Y., Yeow, K.W., Hwang, S.O.: Enhancement of a lightweight attribute-based encryption scheme for the internet of things. IEEE Internet of Things Journal pp. 6384–6395 (2019). <https://doi.org/10.1109/JIOT.2019.2900631>
34. Tseng, Y.F., Huang, J.J.: Cryptanalysis on two pairing-free ciphertext-policy attribute-based encryption schemes. In: 2020 International Computer Symposium (ICS). pp. 403–407 (2020). <https://doi.org/10.1109/ICS51289.2020.00086>
35. Wang, G., Wan, M., Liu, Z., Gu, D.: Fully secure lattice-based ABE from noisy linear functional encryption. In: Yu, Y., Yung, M. (eds.) Information Security and Cryptology. pp. 421–441 (2021)
36. Wang, M., Cao, Z., Dong, X., Du, R., Chen, J.: Decentralized multiauthority kpabe scheme without bilinear pairings. IEEE Internet of Things Journal pp. 726–738 (2025). <https://doi.org/10.1109/JIOT.2024.3470322>
37. Wang, Y., Chen, B., Li, L., Ma, Q., Li, H., He, D.: Efficient and secure ciphertext-policy attribute-based encryption without pairing for cloud-assisted smart grid. IEEE Access pp. 40704–40713 (2020). <https://doi.org/10.1109/ACCESS.2020.2976746>
38. Wang, Z., Fan, X., Liu, F.H.: FE for inner products and its application to decentralized ABE. In: Lin, D., Sako, K. (eds.) Public-Key Cryptography – PKC 2019. pp. 97–127 (2019)
39. Yao, X., Chen, Z., Tian, Y.: A lightweight attribute-based encryption scheme for the internet of things. Future Generation Computer Systems pp. 104–112 (2015). <https://doi.org/10.1016/j.future.2014.10.010>
40. Yao, X., Zhou, J., Du, X., Zhang, S.: A cp-abe and iota-based lightweight sensitive data access control scheme for iot. IEEE Internet of Things Journal pp. 40831–40844 (2024). <https://doi.org/10.1109/JIOT.2024.3456553>

A Syntax of IPFE and its security definition

In the case where the key space is $\mathbb{Z}_q^\ell$, the IPFE scheme is stateful since it must maintain the list of previously key queries as internal state to prevent an adversary from having ℓ independent secret keys allowing to uncover the master secret key [3].

A stateful IPFE scheme computing inner products in $\mathbb{Z}_q$ consists of four polynomial-time algorithms:

1. **Setup**($1^\lambda, 1^\ell$). Input: a security parameter λ and a functionality parameter ℓ . Output: a master public key mpk and a master secret key msk .

2. $\text{KeyGen}(msk, st, \mathbf{x})$. Input: the master secret key msk , an internal state st and a vector $\mathbf{x} \in \mathbb{Z}_q^\ell$. Output: a secret key $sk_{\mathbf{x}}$.
3. $\text{Encrypt}(mpk, \mathbf{y})$. Input: the master public key mpk and a vector $\mathbf{y} \in \mathbb{Z}_q^\ell$. Output: a ciphertext $C_{\mathbf{y}}$.
4. $\text{Decrypt}(mpk, sk_{\mathbf{x}}, C_{\mathbf{y}})$. Input: the master public key mpk , a secret key $sk_{\mathbf{x}}$ and a ciphertext $c_{\mathbf{y}}$. Output: the inner product $\langle \mathbf{x}, \mathbf{y} \rangle \in \mathbb{Z}_q$.

For correctness, it is required that for all $\mathbf{x} \in \mathbb{Z}_q^\ell$ and all $\mathbf{y} \in \mathbb{Z}_q^\ell$, we have $\text{Decrypt}(mpk, sk_{\mathbf{x}}, \text{Encrypt}(mpk, \mathbf{y})) = \langle \mathbf{x}, \mathbf{y} \rangle$ or $\perp$ with negligible probability.

A.1 IND-CPA security

Indistinguishability under Chosen Plaintext Attack security is defined via the following game:

1. **Setup**: the challenger $\mathcal{C}$ runs $\text{Setup}(1^\lambda, 1^\ell)$, obtains (msk, mpk) and gives mpk to the adversary $\mathcal{A}$.
2. **Key query 1**: the adversary $\mathcal{A}$ requests from $\mathcal{C}$ the secret keys associated to vectors $\{\mathbf{x}_i\}_{i=1}^q$, $q \in \text{poly}(\lambda)$ of its choice. These queries can be made adaptively. For each $\mathbf{x}_i$, $\mathcal{C}$ runs $\text{KeyGen}(msk, st, \mathbf{x}_i)$ and gives the result to $\mathcal{A}$.
3. **Challenge**: $\mathcal{A}$ outputs two messages $\mathbf{y}_0$ and $\mathbf{y}_1$ of the same length subject to the restriction that $\langle \mathbf{x}, \mathbf{y}_0 \rangle = \langle \mathbf{x}, \mathbf{y}_1 \rangle$ for all $\mathbf{x}$ queried during the phase **Key query 1**. $\mathcal{C}$ randomly selects $b \in \{0, 1\}$, runs $\text{Encrypt}(mpk, \mathbf{y}_b)$ and sends the result to $\mathcal{A}$.
4. **Key query 2**: key query 1 is repeated with the restriction that $\langle \mathbf{x}, \mathbf{y}_0 \rangle = \langle \mathbf{x}, \mathbf{y}_1 \rangle$ for each query $\mathbf{x}$.
5. **Guess**: $\mathcal{A}$ outputs a guess b' of b and wins if $b' = b$.

The advantage of an adversary $\mathcal{A}$ in this IND-CPA game is defined as

$$\text{Adv}_{\text{IPFE}, \mathcal{A}}^{\text{IND-CPA}}(\lambda) = \Pr[b' = b] - \frac{1}{2}$$

Definition 1. An IPFE scheme is adaptively IND-CPA secure if for any polynomial time adversary $\mathcal{A}$, the function $\text{Adv}_{\text{IPFE}, \mathcal{A}}^{\text{IND-CPA}}(\cdot)$ is negligible.

B Syntax of CP-ABE and its security definition

A ciphertext-policy ABE scheme consists of four polynomial-time algorithms:

1. $\text{Setup}(1^\lambda)$. Input: the security parameter λ . Output: a master public key mpk and a master secret key msk .

2. $\text{KeyGen}(msk, \mathbb{S})$. Input: the master secret key msk and a set of attributes $\mathbb{S}$. Output: a secret key sk .
3. $\text{Encrypt}(mpk, \mathbb{P}, m)$. Input: the master public key mpk , an access policy $\mathbb{P}$ and a message m . Output: a ciphertext C .
4. $\text{Decrypt}(mpk, C, sk)$. Input: the master public key mpk , a ciphertext C and a secret key sk . Output: the plaintext m or $\perp$ meaning that C is malformed or sk does not satisfy the access policy $\mathbb{P}$.

B.1 IND-CPA security for secret policy CP-ABE

We define indistinguishability under chosen plaintext attack security for a secret policy CP-ABE scheme via the following game. The challenger encrypts messages provided by the adversary $\mathcal{A}$ using an access policy $\mathbb{P}$ that is unknown to $\mathcal{A}$.

1. **Setup**: the challenger $\mathcal{C}$ runs $\text{Setup}(1^\lambda)$, obtains (msk, mpk) and gives mpk to the adversary.
2. **Key query 1**: $\mathcal{A}$ requests from $\mathcal{C}$ the secret keys associated to attribute sets $\{\mathbb{S}_i\}_{i=1}^q$, $q \in \text{poly}(\lambda)$ of its choice. These requests can be made adaptively. For each $\mathbb{S}_i$, $\mathcal{C}$ runs $\text{KeyGen}(msk, \mathbb{S}_i)$ and gives the result to $\mathcal{A}$.
3. **Challenge**: $\mathcal{A}$ outputs two distinct messages m_0 and m_1 of the same length. The challenger selects an access policy $\mathbb{P}$ and chooses uniformly a bit $b \in \{0, 1\}$, runs $\text{Encrypt}(mpk, \mathbb{P}, m_b)$ and sends the result to $\mathcal{A}$.
4. **Key query 2**: key query 1 is repeated.
5. **Guess**: $\mathcal{A}$ outputs a guess b' of b and wins if $b' = b$.

The advantage of an adversary $\mathcal{A}$ in this IND-CPA game is defined as

$$\text{Adv}_{\text{CPABE}, \mathcal{A}}^{\text{IND-CPA}}(\lambda) = \Pr[b' = b] - \frac{1}{2}$$

Definition 2. A CP-ABE scheme is adaptively IND-CPA secure if for any polynomial time adversary $\mathcal{A}$, the function $\text{Adv}_{\text{CPABE}, \mathcal{A}}^{\text{IND-CPA}}(\cdot)$ is negligible.

C Syntax of IBE and its security definition

An identity-based encryption scheme consists of four polynomial-time algorithms:

1. $\text{Setup}(1^\lambda)$. Input: the security parameter λ . Output: a master secret key msk and system parameters denoted by $params$.
2. $\text{KeyGen}(params, msk, \text{ID})$. Input: the system parameters $params$, the master secret key msk and an identity $\text{ID} \in \mathbb{Z}_q$. Output: a secret key sk .
3. $\text{Encrypt}(params, \text{ID}, m)$. Input: the system parameters $params$, an identity $\text{ID} \in \mathbb{Z}_q$, and a message m . Output: a ciphertext C .

4. **Decrypt**($params, sk_{ID}, C$). Input: the system parameters $params$, a secret key sk_{ID} and a ciphertext C . Output: the plaintext m or $\perp$ meaning that C is malformed or is not obtained using ID .

C.1 IND-CPA security

Boneh et al. [7] defined IND-CPA security for IBE via the following security game between an adversary $\mathcal{A}$ and a challenger $\mathcal{C}$:

1. **Setup**: $\mathcal{C}$ runs $\text{Setup}(1^\lambda)$, obtains $(params, msk)$ and gives $params$ to $\mathcal{A}$.
2. **Key query 1**: Adversary $\mathcal{A}$ requests from $\mathcal{C}$ the secret keys associated to identities $\{ID_i\}_{i=1}^q$, $q \in \text{poly}(\lambda)$ of its choice. These requests can be made adaptively. For each ID_i , $\mathcal{C}$ runs $\text{KeyGen}(params, msk, ID_i)$ and gives the result to $\mathcal{A}$.
3. **Challenge**: $\mathcal{A}$ outputs two distinct messages m_0, m_1 of the same length and an identity ID^* subject to the restriction that no secret key has been previously generated for it. $\mathcal{C}$ randomly selects $b \in \{0, 1\}$, runs $\text{Encrypt}(params, ID^*, m_b)$ and sends the result to $\mathcal{A}$.
4. **Key query 2**: key query 1 is repeated with the restriction that $ID_i \neq ID^*$.
5. **Guess**: $\mathcal{A}$ outputs a guess b' of b and wins if $b' = b$.

The advantage of an adversary $\mathcal{A}$ in this IND-CPA game is defined as

$$\text{Adv}_{\text{IBE}, \mathcal{A}}^{\text{IND-CPA}}(\lambda) = \Pr[b' = b] - \frac{1}{2}$$

Definition 3. An IBE scheme is adaptively IND-CPA secure if for any polynomial time adversary $\mathcal{A}$, the function $\text{Adv}_{\text{IBE}, \mathcal{A}}^{\text{IND-CPA}}(\cdot)$ is negligible.

D Proof of Theorem 1

Assume there exists an IND-CPA adversary $\mathcal{A}$ that has non-negligible advantage against our CP-ABE scheme. We show below how an adversary $\mathcal{B}$ would interact with $\mathcal{A}$ in five phases to break the IND-CPA security of the underlying IPFE scheme. Let $\mathcal{C}$ be the challenger of $\mathcal{B}$.

1. **Setup**: $\mathcal{C}$ runs $(msk, mpk) \leftarrow \text{IPFE.Setup}(1^\lambda, 1^\ell)$ and gives mpk to $\mathcal{B}$. The latter chooses a cyclic group $\mathbb{G}$ of prime order $q > 2^\lambda$ with generator g and gives to $\mathcal{A}$ the tuple $(\mathbb{G}, g, mpk)$.
2. **Key query 1**: $\mathcal{A}$ makes repeated secret key queries to $\mathcal{B}$. When $\mathcal{B}$ receives one of these requests associated with a set of attributes $\mathbb{S} \in (\mathbb{Z}_q^* \times [\ell])^n$ where $n \leq \ell$, it does the following:
 - converts $\mathbb{S}$ to a vector $\mathbf{S} = (s_1, \dots, s_\ell) \in \mathbb{Z}_q^\ell$.

- sends to $\mathcal{C}$ a secret key request for $\mathcal{S}$ to obtain $sk_{\mathcal{S}}$. By the way, $\mathcal{C}$ runs $sk_{\mathcal{S}} \leftarrow \text{IPFE.KeyGen}(msk, st, \mathcal{S})$ to respond to this query.
 - returns $sk_{\mathcal{S}}$ to $\mathcal{A}$.
3. **Challenge:** $\mathcal{A}$ outputs two distinct messages $m_0, m_1 \in \mathbb{G}$ of the same length. $\mathcal{B}$ chooses an access policy $\mathbb{P} \in (\mathbb{Z}_q^* \times [\ell])^n$ where $n \leq \ell$, converts it to a vector $\mathbf{P} \in \mathbb{Z}_q^\ell$. $\mathcal{B}$ also chooses two random vectors $\mathbf{r}_0, \mathbf{r}_1 \in (\mathbb{Z}_q^*)^\ell$, computes $\mathbf{y}_0 = \mathbf{P} \odot \mathbf{r}_0$, $\mathbf{y}_1 = \mathbf{P} \odot \mathbf{r}_1$ and sends $\mathbf{y}_0, \mathbf{y}_1$ as challenge messages to $\mathcal{C}$. The latter randomly selects $b \in \{0, 1\}$, computes $c_2 \leftarrow \text{IPFE.Encrypt}(mpk, \mathbf{y}_b)$ and sends the result to $\mathcal{B}$. Upon receiving c_2 , $\mathcal{B}$ chooses $b' \in \{0, 1\}$, computes $c_1 = m_{b'} g^{(\mathbf{P}, \mathbf{y}_{b'})}$ and returns (c_1, c_2) to $\mathcal{A}$ as the challenge ciphertext.
 4. **Key query 2:** Similar to Key query 1.
 5. **Guess:** $\mathcal{A}$ outputs a guess b^* . $\mathcal{B}$ outputs the same guess as $\mathcal{A}$.

When $b = b'$ adversary $\mathcal{B}$ simulates perfectly for $\mathcal{A}$ the challenger during a chosen plaintext attack against our CP-ABE scheme. This happens with probability $1/2$ since b is independent of b' . When $b \neq b'$ $\mathcal{B}$ produces an invalid ciphertext and $\mathcal{A}$ can do nothing but guess which of the two messages was encrypted. Therefore, we have

$$\text{Adv}_{\text{CPABE}, \mathcal{A}}^{\text{IND-CPA}} = 2 \cdot \text{Adv}_{\text{IPFE}, \mathcal{B}}^{\text{IND-CPA}}$$

Since the IPFE scheme is IND-CPA secure, $\text{Adv}_{\text{IPFE}, \mathcal{B}}^{\text{IND-CPA}}$ is negligible. Thus, the IND-CPA advantage of $\mathcal{A}$ against our CP-ABE scheme is negligible. This concludes the proof.

E IPFE-DDH^{*} : the modified version of the DDH based IPFE scheme of Agrawal *et al.*

The original version of the DDH based IPFE scheme is available in [3]. The modification only concerns the decryption algorithm. It returns $g_1^{\langle \mathbf{x}, \mathbf{y} \rangle}$ instead of $\langle \mathbf{x}, \mathbf{y} \rangle$ thus avoiding the computation of the discrete logarithm which is too expensive.

$\text{Setup}(1^\lambda, 1^\ell)$

1. Choose a cyclic group $\mathbb{G}$ of prime order $q > 2^\lambda$ with generators g_1, g_2 .
2. Choose 2 vectors $\mathbf{u} = (u_1, \dots, u_\ell) \xleftarrow{R} \mathbb{Z}_q^\ell$, $\mathbf{v} = (v_1, \dots, v_\ell) \xleftarrow{R} \mathbb{Z}_q^\ell$.
3. For each $i \in [\ell]$ compute $h_i = g_1^{u_i} \cdot g_2^{v_i}$.
4. Return $msk = (\mathbf{u}, \mathbf{v})$, $mpk = (\mathbb{G}, g_1, g_2, \{h_i\}_{i=1}^\ell)$.

$\text{KeyGen}(msk, st, \mathbf{x} \in \mathbb{Z}_q^\ell)$. The internal state st contains at most ℓ tuples of the form $(\mathbf{z}_i, \mathbf{z}'_i, u_{\mathbf{z}_i}, v_{\mathbf{z}_i})$ where $(\mathbf{z}'_i, u_{\mathbf{z}_i}, v_{\mathbf{z}_i})$ are previously generated secret keys for $\mathbf{z}_i \in \mathbb{Z}_q^\ell$. To generate the j^{th} secret key, this algorithm does the following:

- checks if $\mathbf{x}$ is linearly independent of the $\mathbf{z}_i$'s:
 If $\nexists \{\gamma_i\}_{i=1}^{j-1} \in \mathbb{Z}^{j-1}$ such that $\mathbf{x} = \sum_{i=1}^{j-1} \gamma_i \mathbf{z}_i \pmod q$ then

set $\mathbf{x}' = \mathbf{x}$, $u_{\mathbf{x}} = \langle \mathbf{u}, \mathbf{x} \rangle$, $v_{\mathbf{x}} = \langle \mathbf{v}, \mathbf{x} \rangle$ and add $(\mathbf{x}, \mathbf{x}', u_{\mathbf{x}}, v_{\mathbf{x}})$ to st .
 Else set
 $\mathbf{x}' = \sum_{i=1}^{j-1} \gamma_i \mathbf{z}'_i \in \mathbb{Z}^\ell$, $u_{\mathbf{x}} = \sum_{i=1}^{j-1} \gamma_i u_{\mathbf{z}_i} \in \mathbb{Z}$ and $v_{\mathbf{x}} = \sum_{i=1}^{j-1} \gamma_i v_{\mathbf{z}_i} \in \mathbb{Z}$.
 – returns $sk_{\mathbf{x}} = (\mathbf{x}', u_{\mathbf{x}}, v_{\mathbf{x}})$.

Encrypt($mpk, \mathbf{y} = (y_1, \dots, y_\ell) \in \mathbb{Z}_q^\ell$)

1. Pick $r \xleftarrow{R} \mathbb{Z}_q^*$.
2. Compute $C = g_1^r$, $D = g_2^r$, $\{E_i = g_1^{y_i} \cdot h_i^r\}_{i=1}^\ell$.
3. Return $C_{\mathbf{y}} = (C, D, E_1, \dots, E_\ell)$.

Decrypt($mpk, sk_{\mathbf{x}} = (\mathbf{x}', u_{\mathbf{x}}, v_{\mathbf{x}}), C_{\mathbf{y}}$)

- $\mathbf{x}' = (x'_1, \dots, x'_\ell) \in \mathbb{Z}_q^\ell$
1. Return $E_{\mathbf{x}} = (\prod_{i=1}^\ell E_i^{x'_i}) / (C^{u_{\mathbf{x}}} \cdot D^{v_{\mathbf{x}}}) = g_1^{\langle \mathbf{x}, \mathbf{y} \rangle}$.

F Proof of Theorem 2

Assume there exists an IND-CPA adversary $\mathcal{A}$ that has non-negligible advantage against our SIS-IBE scheme. We show below how an adversary $\mathcal{B}$ would interact with $\mathcal{A}$ in five phases to break the IND-CPA security of the underlying IPFE scheme. Let $\mathcal{C}$ be the challenger of $\mathcal{B}$.

1. **Setup:** $\mathcal{C}$ runs $(msk, mpk) \leftarrow \text{IPFE.Setup}(1^\lambda, 1^\ell)$ and gives mpk to $\mathcal{B}$. The latter chooses a cyclic group $\mathbb{G}$ of prime order $q > 2^\lambda$ with generator g , creates a list $\mathbb{L}$ (initially empty) of vectors in $(\mathbb{Z}_q^*)^\ell$ that has a maximum capacity of $\ell - 1$ vectors, sets $params = (mpk, \mathbb{G}, g, \mathbb{L})$ and gives $params$ to $\mathcal{A}$.
2. **Key query 1:** $\mathcal{A}$ makes repeated secret key queries associated to identities $\{\text{ID}_i\}_{i=1}^q$, $q \leq \ell - 2$ of its choice. Assume $\mathcal{A}$ does not make different queries for the same ID. When $\mathcal{B}$ receives one of these requests associated to an identity ID, it does the following:
 - picks a random $\mathbf{x}_{\text{ID}} \in (\mathbb{Z}_q^*)^\ell$ such that the set of vectors $\{\mathbf{x}\}_{\mathbf{x} \in \mathbb{L}} \cup \{\mathbf{x}_{\text{ID}}\}$ is linearly independent, and adds $\mathbf{x}_{\text{ID}}$ to $\mathbb{L}$ (at any time $\mathcal{A}$ gets an update on $\mathbb{L}$).
 - sends to $\mathcal{C}$ a secret key request for $\mathbf{x}_{\text{ID}}$ and obtains $sk_{\mathbf{x}_{\text{ID}}}$. By the way, $\mathcal{C}$ runs $sk_{\mathbf{x}_{\text{ID}}} \leftarrow \text{IPFE.KeyGen}(msk, \mathbf{x}_{\text{ID}})$ to respond to this query.
 - returns $sk_{\mathbf{x}_{\text{ID}}}$ to $\mathcal{A}$.
3. **Challenge:** $\mathcal{A}$ outputs two distinct messages $m_0, m_1 \in \mathbb{G}$ of the same length and an identity ID^* that has not been associated to a query in the previous phase. $\mathcal{B}$ picks a random $\mathbf{x}_{\text{ID}^*} \in (\mathbb{Z}_q^*)^\ell$ such that the set of vectors $\{\mathbf{x}\}_{\mathbf{x} \in \mathbb{L}} \cup \{\mathbf{x}_{\text{ID}^*}\}$ is linearly independent, chooses two random vectors $\mathbf{r}_0, \mathbf{r}_1 \in (\mathbb{Z}_q^*)^\ell$, computes $\mathbf{y}_0 = \mathbf{x}_{\text{ID}^*} \odot \mathbf{r}_0$, $\mathbf{y}_1 = \mathbf{x}_{\text{ID}^*} \odot \mathbf{r}_1$ and sends $\mathbf{y}_0, \mathbf{y}_1$ as challenge messages to $\mathcal{C}$. The latter randomly selects $b \in \{0, 1\}$, computes

$c_2 \leftarrow \text{IPFE.Encrypt}(mpk, \mathbf{y}_b)$ and sends the result to $\mathcal{B}$. Upon receiving c_2 , $\mathcal{B}$ chooses $b' \in \{0, 1\}$, computes $c_1 = m_{b'} g^{\langle x_{\text{ID}^*}, \mathbf{y}_{b'} \rangle}$ and returns (c_1, c_2) to $\mathcal{A}$ as the challenge ciphertext.

4. **Key query 2:** Similar to Key query 1 with identities $\text{ID}_i \neq \text{ID}^*$. In total, $\mathcal{A}$ makes at most $\ell - 2$ key queries.
5. **Guess:** $\mathcal{A}$ outputs a guess b^* . $\mathcal{B}$ outputs the same guess as $\mathcal{A}$.

When $b = b'$ adversary $\mathcal{B}$ simulates perfectly for $\mathcal{A}$ the challenger during a chosen plaintext attack against our CP-ABE scheme. This happens with probability $1/2$ since b is independent of b' . When $b \neq b'$ $\mathcal{B}$ produces an invalid ciphertext and $\mathcal{A}$ can do nothing but guess which of the two messages was encrypted. Therefore, we have

$$\text{Adv}_{\text{SIS-IBE}, \mathcal{A}}^{\text{IND-CPA}} = 2 \cdot \text{Adv}_{\text{IPFE}, \mathcal{B}}^{\text{IND-CPA}}$$

Since the IPFE scheme is IND-CPA secure, $\text{Adv}_{\text{IPFE}, \mathcal{B}}^{\text{IND-CPA}}$ is negligible. Thus, the IND-CPA advantage of $\mathcal{A}$ against our SIS-IBE scheme is negligible. This concludes the proof.